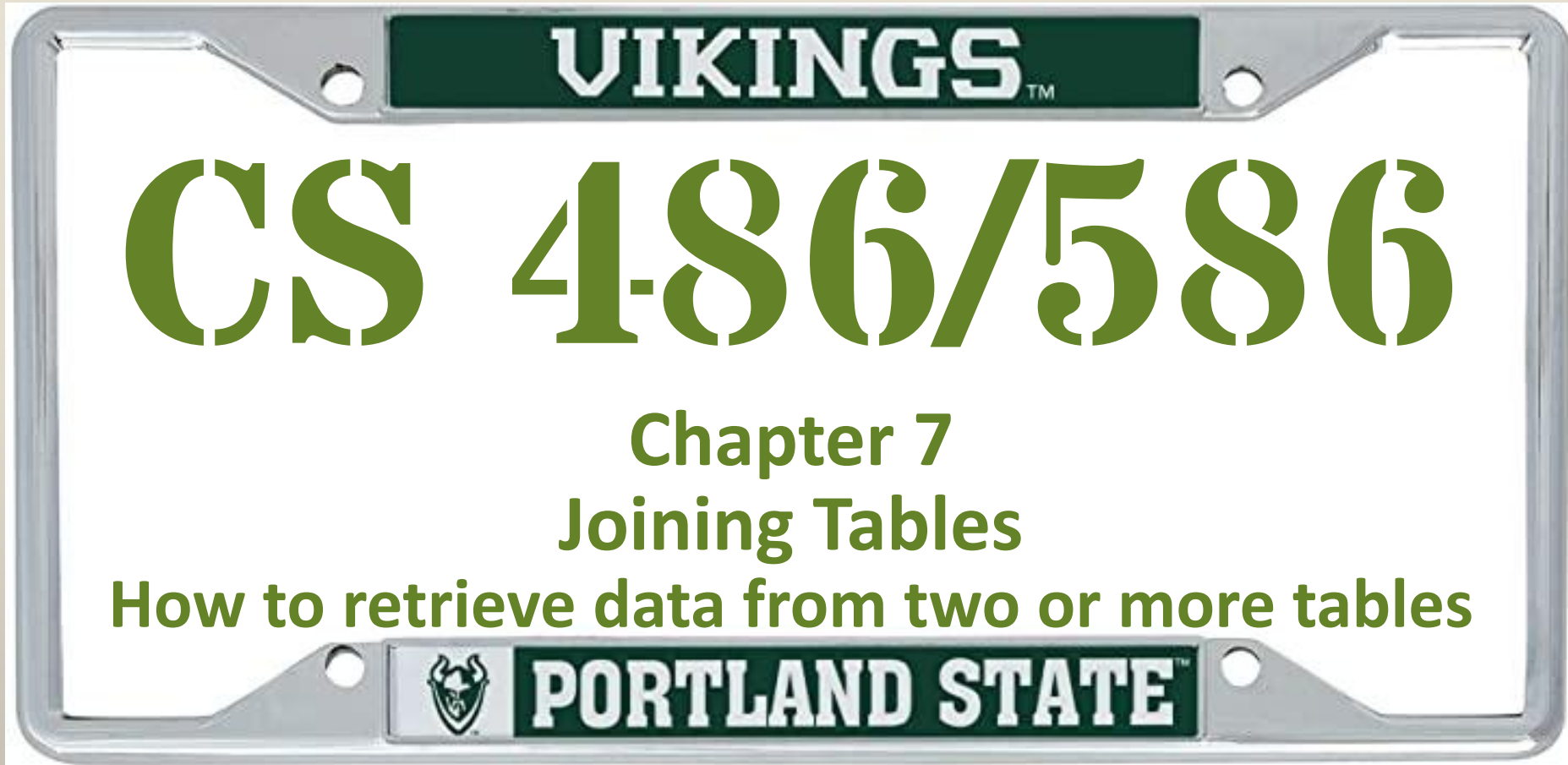




Credit and thank you go to Professor R.Jesse Chaney from Portland State University for generously providing me with this material.

Using **joins**, you retrieve data from two or more tables based on logical **relationships** between the tables.

**Joins indicate how data from one table is used to select the rows in another table.**

The **relationships** may be explicitly defined in the database or may only be implicit as part of a query (**ad hoc**).

A **join condition** defines the way two tables are **related** in a query by:

- Specifying the column (or columns) from each table to be used for the join.
  - A typical join condition specifies a **foreign key from one table** and its **associated (primary) key** in the other table.
- Specifying a logical operator (for example, = or <>,) to be used in comparing values from the columns.

# Terms to know about joins

- Join
- Join condition
- Inner join
- Ad hoc relationship
- Table alias
- Schema
- **Self-join**
- **Explicit syntax**
- **Implicit syntax**

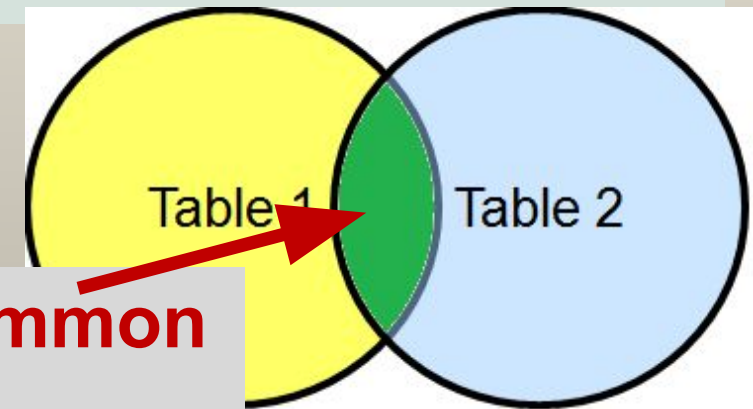
These are the **columns used to join the tables** in the query.

There are additional types of joins.

A relationship used in a query that is **not expressed in the database**.

You can think of a schema **like a directory** containing objects.

Joining a table with itself.



The **inner join** is what is in **common** between the tables.

There are 5 types of standard **ANSI standard joins**:

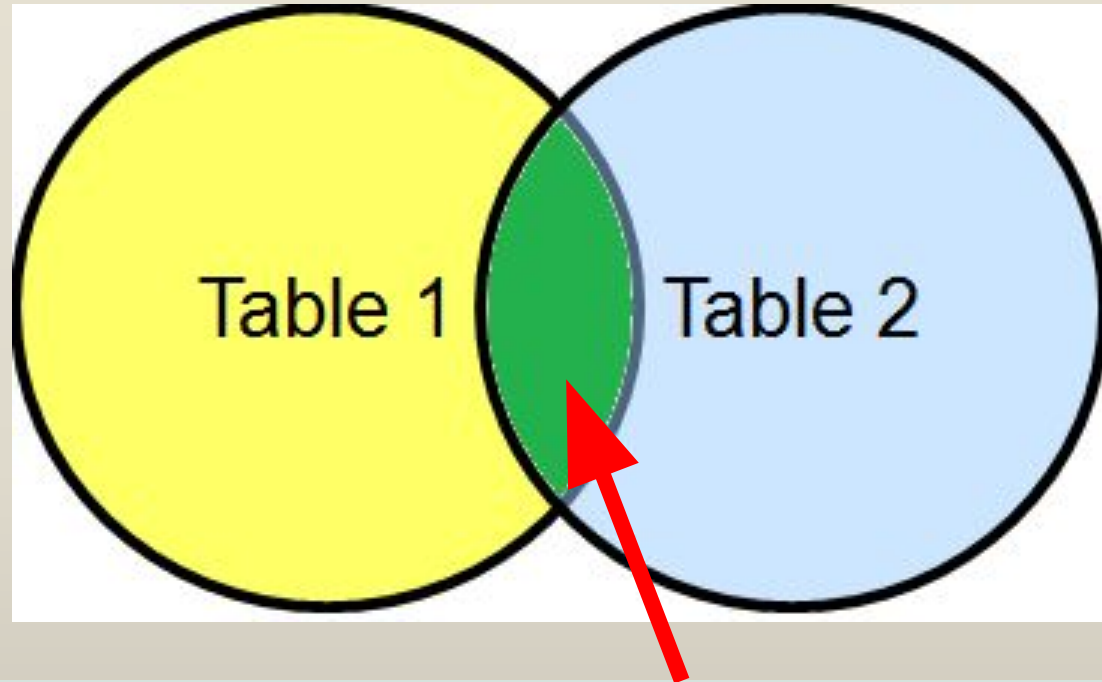
1. **INNER JOIN** (the default, typically just **JOIN**)
2. **LEFT OUTER JOIN** (typically just **LEFT JOIN**)
3. **RIGHT OUTER JOIN** (typically just **RIGHT JOIN**)
4. **FULL OUTER JOIN** (typically just **FULL JOIN**)
5. **CROSS JOIN** (typically just **CROSS**)

You will be asked to list and diagram these **several times** during the term.

You might as well imprint them on your mind now.



# The **INNER** Join



This is the result set. It is what is in **common** between the tables.

The **result set** consists of the rows from both tables that match the join conditions.

# Joins

**Inner joins** return rows only when there is **at least one row from both tables** that matches the join condition.

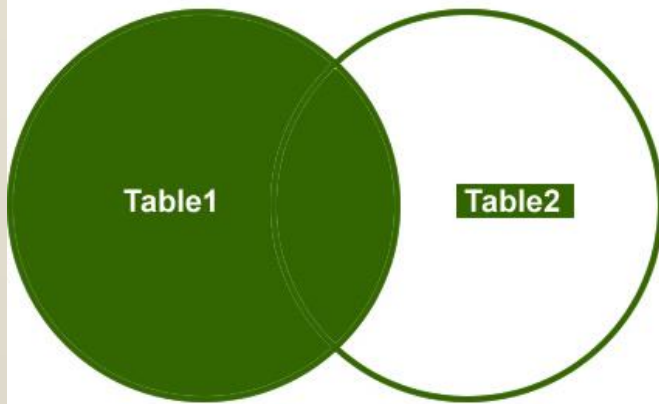
- Inner joins **eliminate the rows that do not match with a row from the other table.**

**Outer joins** return **all rows from at least one of the tables** or views mentioned in the FROM clause, as long as those rows meet any WHERE or HAVING search conditions.

- **All rows are retrieved from the left table referenced with a left outer join, and all rows from the right table referenced in a right outer join.**
- **All rows from both tables are returned in a full outer join.**

# Outer Joins, Venn Diagrams

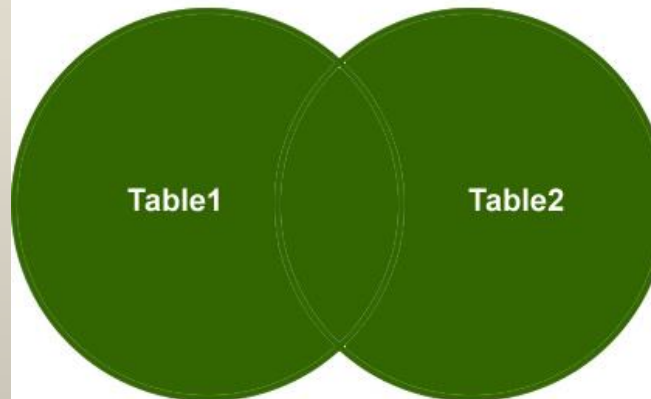
LEFT OUTER JOIN



```
SELECT *  
FROM Table1 t1  
LEFT OUTER JOIN Table2 t2  
ON t1.Col1 = t2.Col1
```

A **left outer join** returns **all the rows from the left table** and rows matching the join condition from the right table.

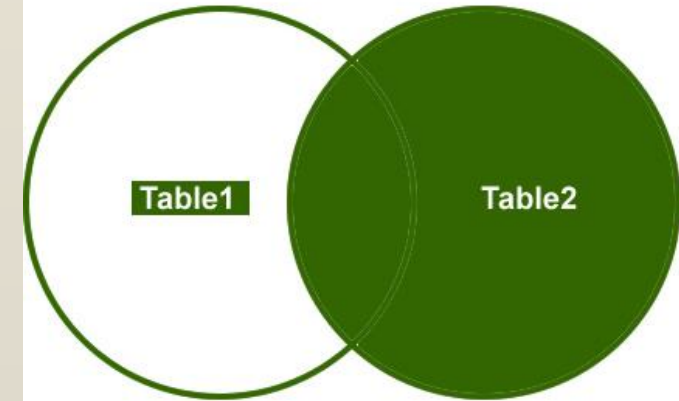
FULL OUTER JOIN



```
SELECT *  
FROM Table1 t1  
FULL OUTER JOIN Table2 t2  
ON t1.Col1 = t2.Col1
```

A **full outer join**

RIGHT OUTER JOIN

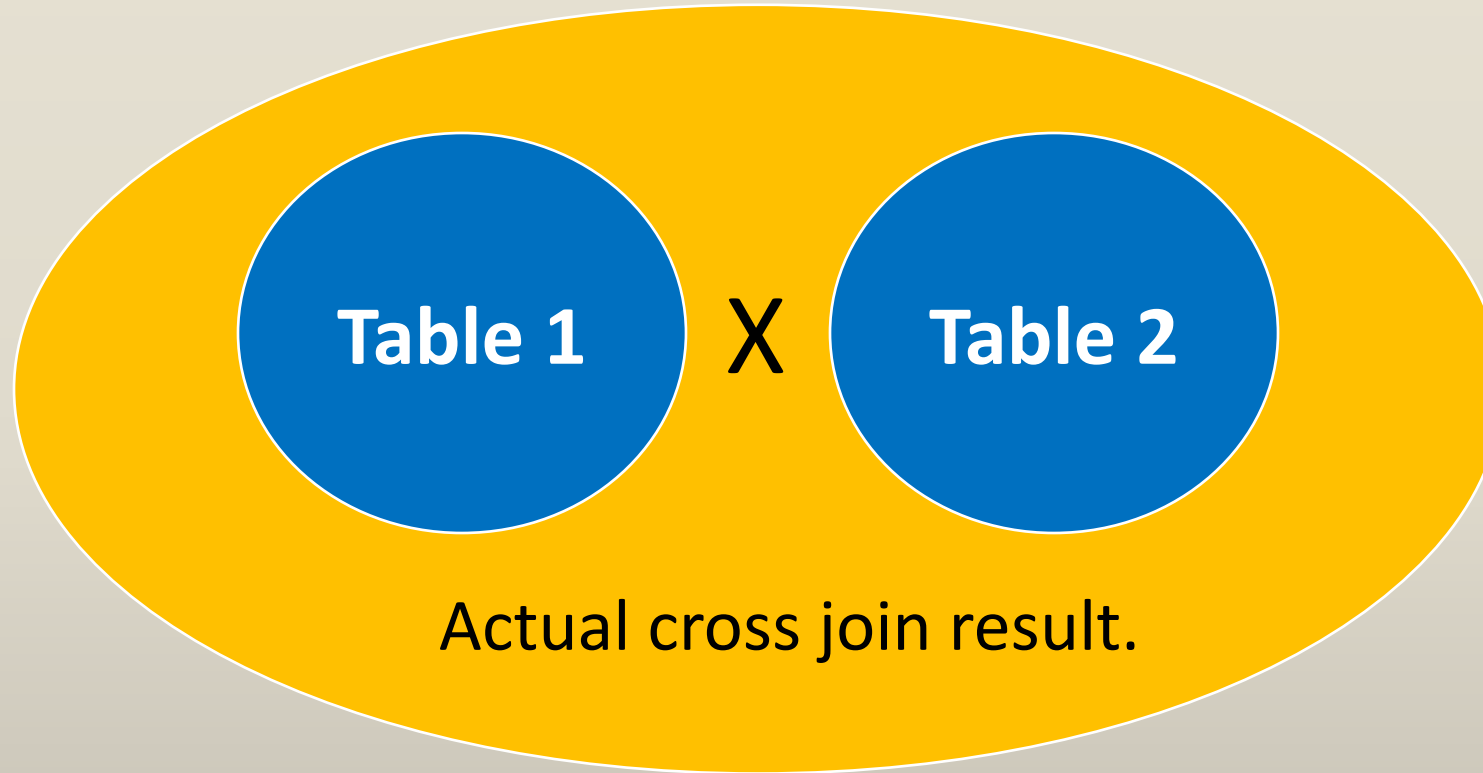


```
SELECT *  
FROM Table1 t1  
RIGHT OUTER JOIN Table2 t2  
ON t1.Col1 = t2.Col1
```

A **right outer join** returns **all the rows from the right table** and rows matching the join condition from the left table.



# The **CROSS** Join



The tables in a **CROSS JOIN** don't need to have any rows that match a join condition.

The **CROSS JOIN** matches each value in the left table to each value in the right table, a cartesian product of the 2 tables.

# Linking Tables Using JOIN

## Common syntax

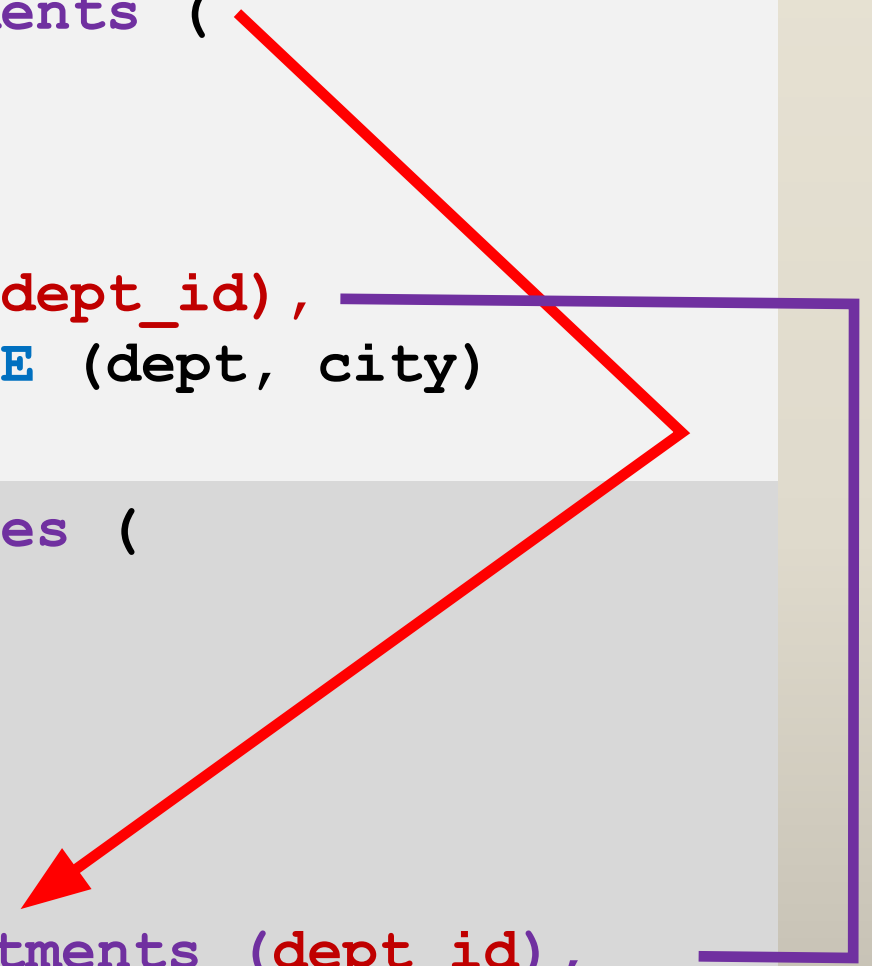
```
SELECT *  
FROM table_a  
JOIN table_b  
ON table_a.key_column =  
table_b.foreign_key_column;
```

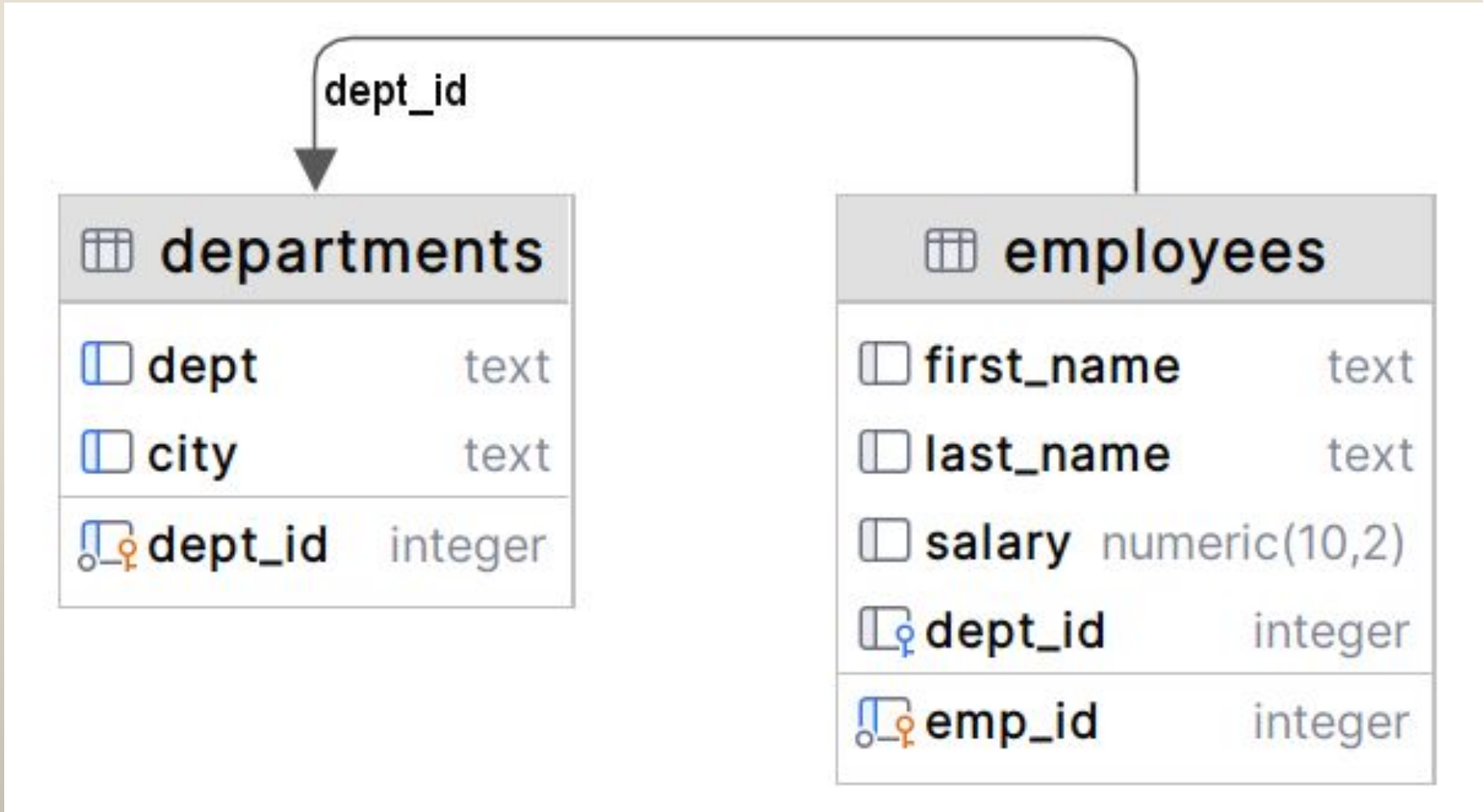
The join condition, defaults to inner

This is the **explicit syntax**

```
CREATE TABLE chap_07_examples.departments (  
    dept_id integer,  
    dept text,  
    city text,  
    CONSTRAINT dept_key PRIMARY KEY (dept_id),  
    CONSTRAINT dept_city_unique UNIQUE (dept, city)  
);
```

```
CREATE TABLE chap_07_examples.employees (  
    emp_id integer,  
    first_name text,  
    last_name text,  
    salary numeric(10,2),  
    dept_id integer  
    REFERENCES chap_07_examples.departments (dept_id),  
    CONSTRAINT emp_key PRIMARY KEY (emp_id)  
);
```





```
INSERT INTO chap_07_examples.departments  
VALUES
```

```
  (1, 'Tax', 'Atlanta'),
```

```
  (2, 'IT', 'Boston');
```

```
INSERT INTO chap_07_examples.employees  
VALUES
```

```
  (1, 'Julia', 'Reyes', 115300,
```

```
  (2, 'Janet', 'King', 98000,
```

```
  (3, 'Arthur', 'Pappas', 72700,
```

```
  (4, 'Michael', 'Taylor', 89500,
```

```
  1),
```

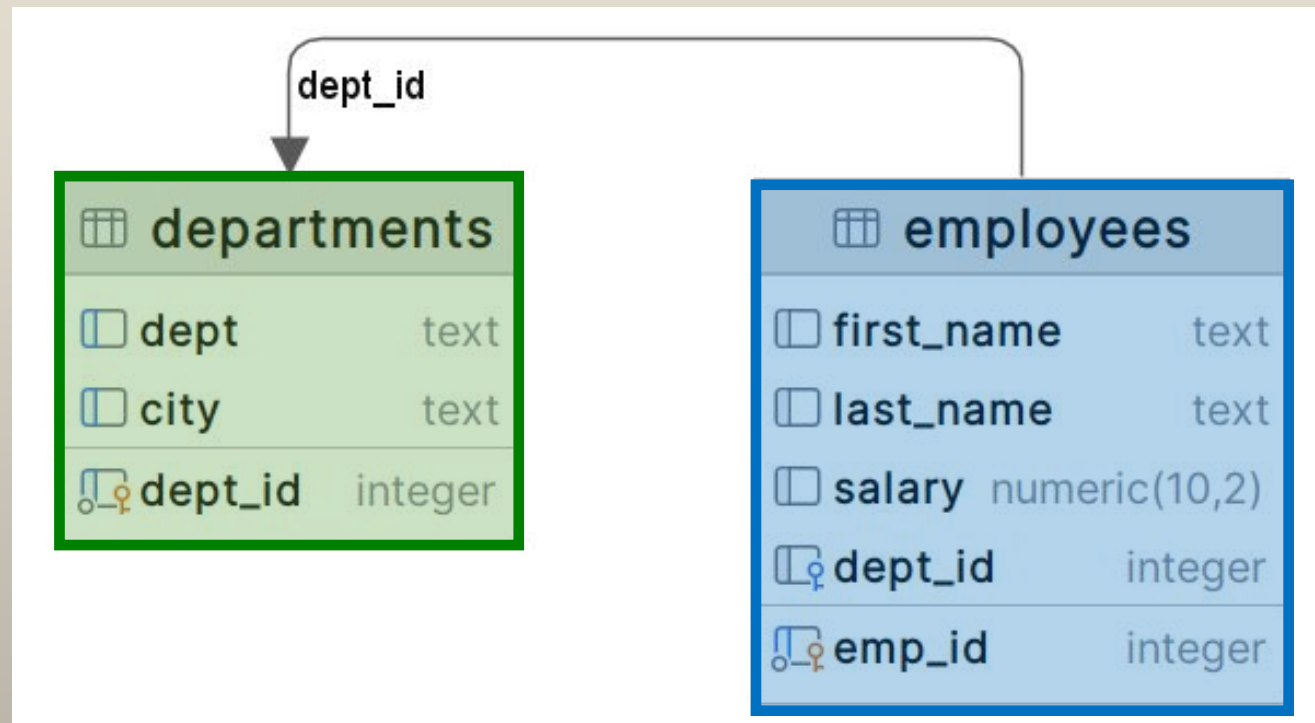
```
  1),
```

```
  2),
```

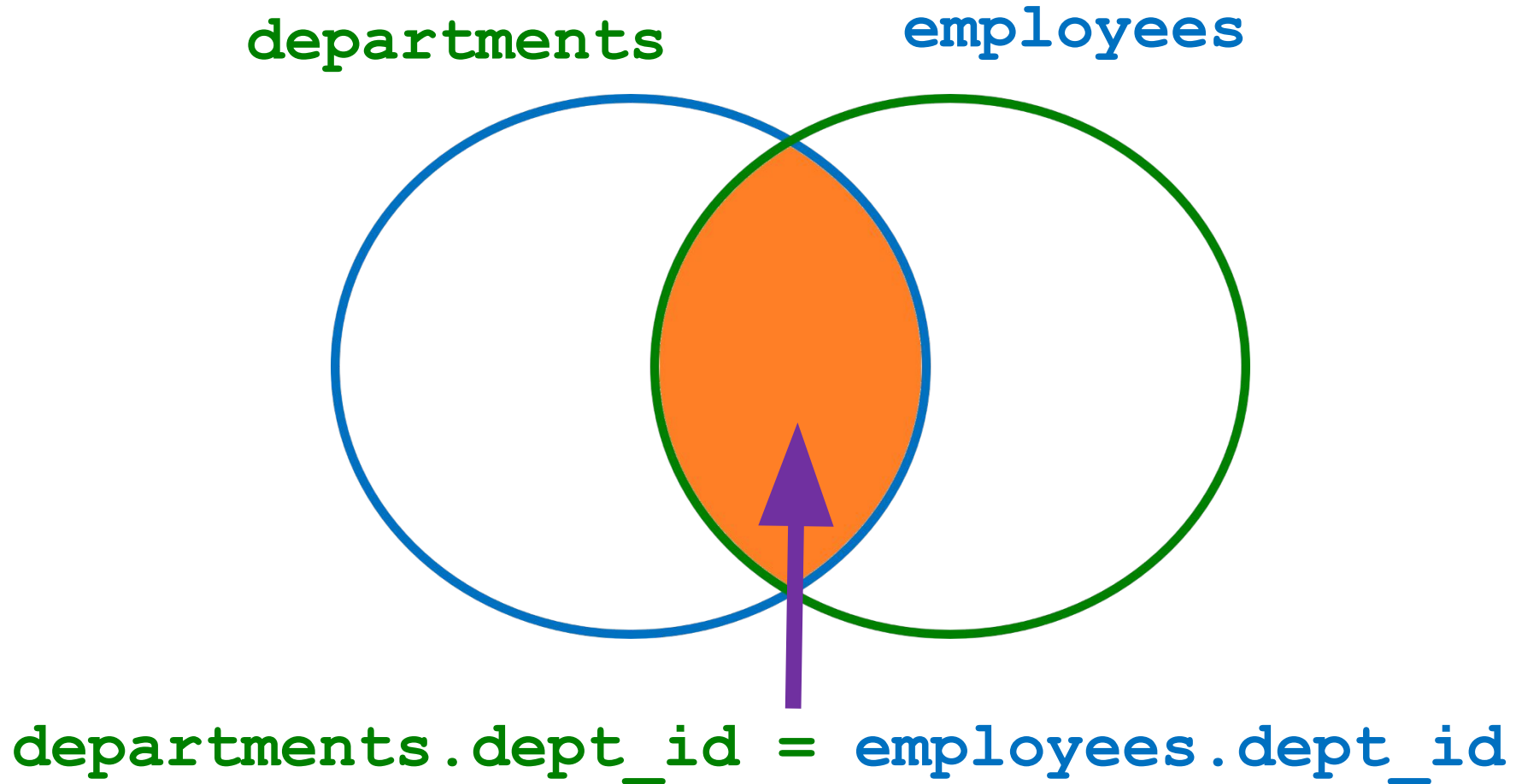
```
  2);
```

The pk from the  
departments table

```
SELECT *  
FROM chap_07_examples.departments  
    JOIN chap_07_examples.employees  
        ON departments.dept_id = employees.dept_id  
ORDER BY employees.dept_id;
```



# The INNER JOIN of employees and departments



```
SELECT *  
FROM chap_07_examples.departments  
  JOIN chap_07_examples.employees  
  ON departments.dept_id = employees.dept_id  
ORDER BY employees.dept_id;
```

A JOIN defaults to INNER JOIN

| emp_id | first_name | last_name | salary  | dept_id | dept_id | dept | city    |
|--------|------------|-----------|---------|---------|---------|------|---------|
| 1      | Julia      | Reyes     | 115,300 | 1       | 1       | Tax  | Atlanta |
| 2      | Janet      | King      | 98,000  | 1       | 1       | Tax  | Atlanta |
| 3      | Arthur     | Pappas    | 72,700  | 2       | 2       | IT   | Boston  |
| 4      | Michael    | Taylor    | 89,500  | 2       | 2       | IT   | Boston  |



```
CREATE TABLE IF NOT EXISTS chap_07_examples.district_2020
(
    id integer CONSTRAINT id_key_2020 PRIMARY KEY,
    school_2020 text
);
```

```
CREATE TABLE IF NOT EXISTS chap_07_examples.district_2035
(
    id integer CONSTRAINT id_key_2035 PRIMARY KEY,
    school_2035 text
);
```

No explicit relationship is declared.

```
INSERT INTO chap_07_examples.district_2020 VALUES
```

```
(1, 'Oak Street School'),  
(2, 'Roosevelt High School'),  
(5, 'Dover Middle School'),  
(6, 'Webutuck High School');
```

```
INSERT INTO chap_07_examples.district_2035 VALUES
```

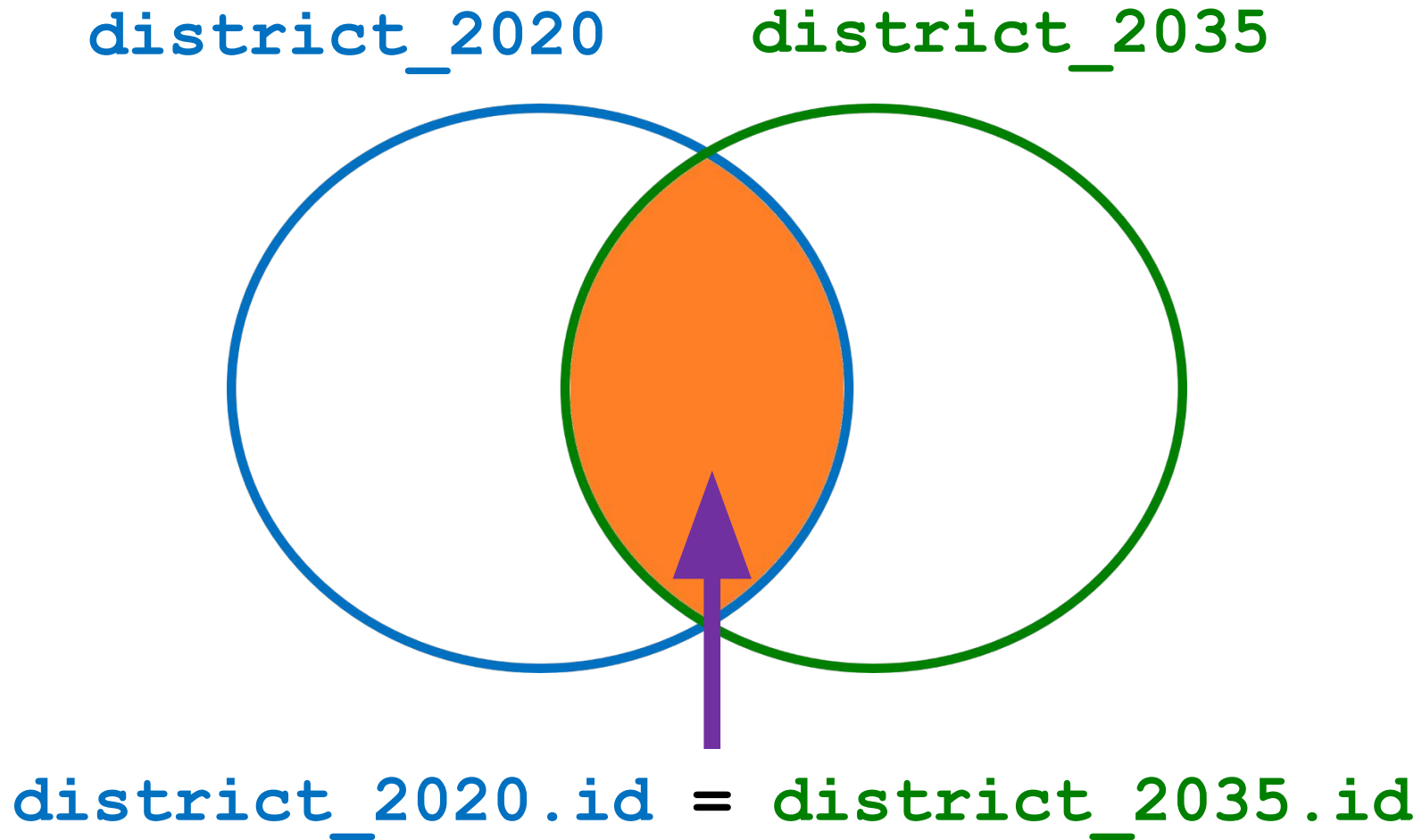
```
(1, 'Oak Street School'),  
(2, 'Roosevelt High School'),  
(3, 'Morrison Elementary'),  
(4, 'Chase Magnet Academy'),  
(6, 'Webutuck High School');
```

No explicit relationship is declared, but there is an implicit relationship within the data.

```
SELECT *  
FROM chap_07_examples.district_2020  
    JOIN chap_07_examples.district_2035  
    ON district_2020.id = district_2035.id  
ORDER BY district_2020.id;
```

| id | school_2020           | id | school_2035           |
|----|-----------------------|----|-----------------------|
| 1  | Oak Street School     | 1  | Oak Street School     |
| 2  | Roosevelt High School | 2  | Roosevelt High School |
| 6  | Webutuck High School  | 6  | Webutuck High School  |

# The **INNER JOIN** of district\_2020 and district\_2035



# The implicit syntax for an **INNER JOIN**

```
SELECT *  
FROM chap_07_examples.district_2020  
      , chap_07_examples.district_2035  
WHERE district_2020.id = district_2035.id  
ORDER BY district_2020.id;
```

| id | school_2020           | id | school_2035           |
|----|-----------------------|----|-----------------------|
| 1  | Oak Street School     | 1  | Oak Street School     |
| 2  | Roosevelt High School | 2  | Roosevelt High School |

**Unless I specifically request the implicit syntax, you must use the explicit syntax. I will count the implicit syntax as incorrect.**

## Explicit syntax

```
SELECT *  
FROM chap_07_examples.district_2020  
    JOIN chap_07_examples.district_2035  
    ON district_2020.id = district_2035.id  
ORDER BY district_2020.id;
```

## Implicit syntax

```
SELECT *  
FROM chap_07_examples.district_2020  
    , chap_07_examples.district_2035  
WHERE district_2020.id = district_2035.id  
ORDER BY district_2020.id;
```

**SELECT \***  
**FROM** chap\_07\_examples.district\_2  
**INNER JOIN** chap\_07\_examples.district\_2035  
**ON** district\_2020.id = district\_2035.id  
**ORDER BY** district\_2020.id;

If you don't specify the type of join, the default is to use an `INNER JOIN`.

| id | school_2020           | id | school_2035           |
|----|-----------------------|----|-----------------------|
| 1  | Oak Street School     | 1  | Oak Street School     |
| 2  | Roosevelt High School | 2  | Roosevelt High School |
| 6  | Webutuck High School  | 6  | Webutuck High School  |

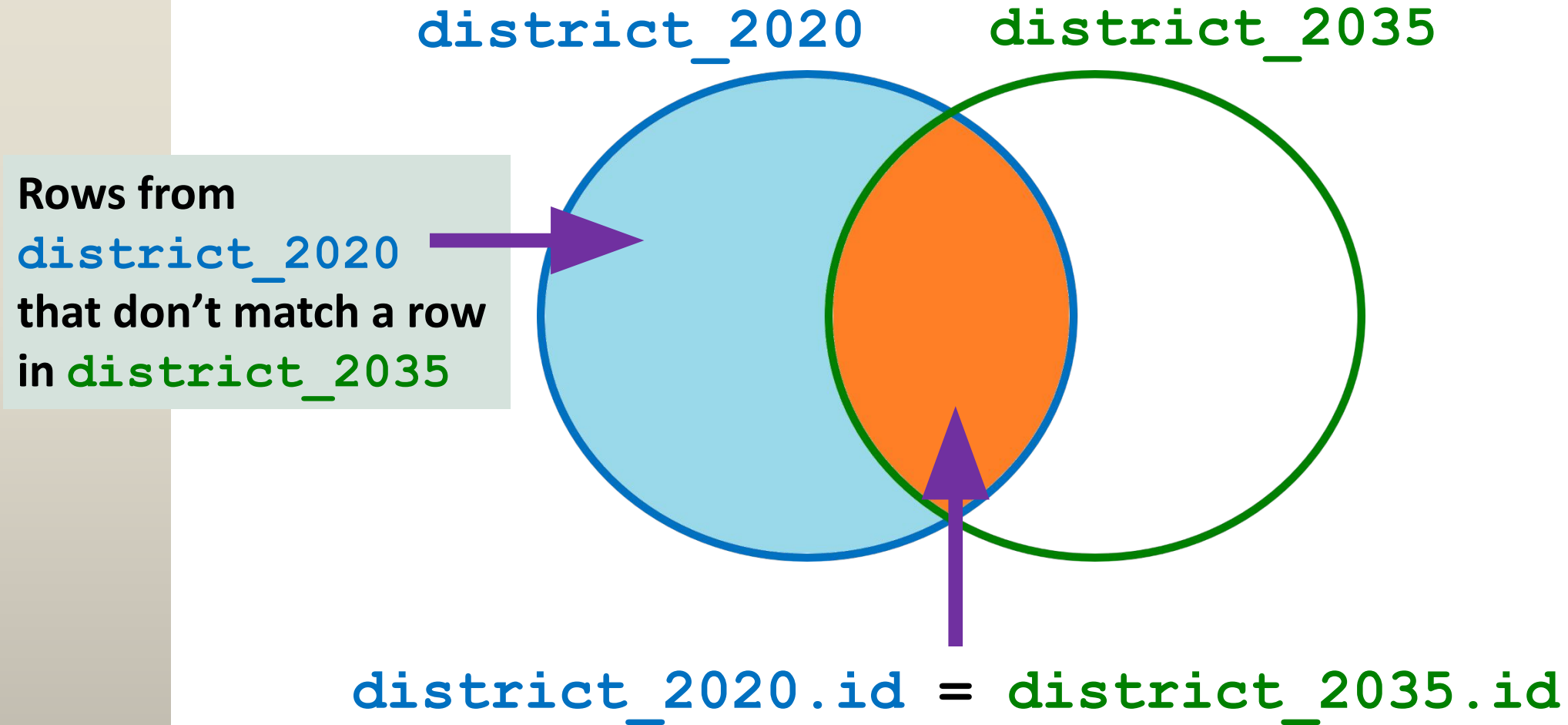
If you're using **identical column names** in a join's **ON** clause, you can reduce redundant output and simplify the query syntax by substituting a **USING** clause in place of the **ON** clause

```
SELECT *  
FROM chap_07_examples.district_2020  
JOIN chap_07_examples.district_2035  
  USING (id)  
ORDER BY district_2020.id;
```

| id | school_2020           | school_2035           |
|----|-----------------------|-----------------------|
| 1  | Oak Street School     | Oak Street School     |
| 2  | Roosevelt High School | Roosevelt High School |
| 6  | Webutuck High School  | Webutuck High School  |



# A LEFT OUTER Join



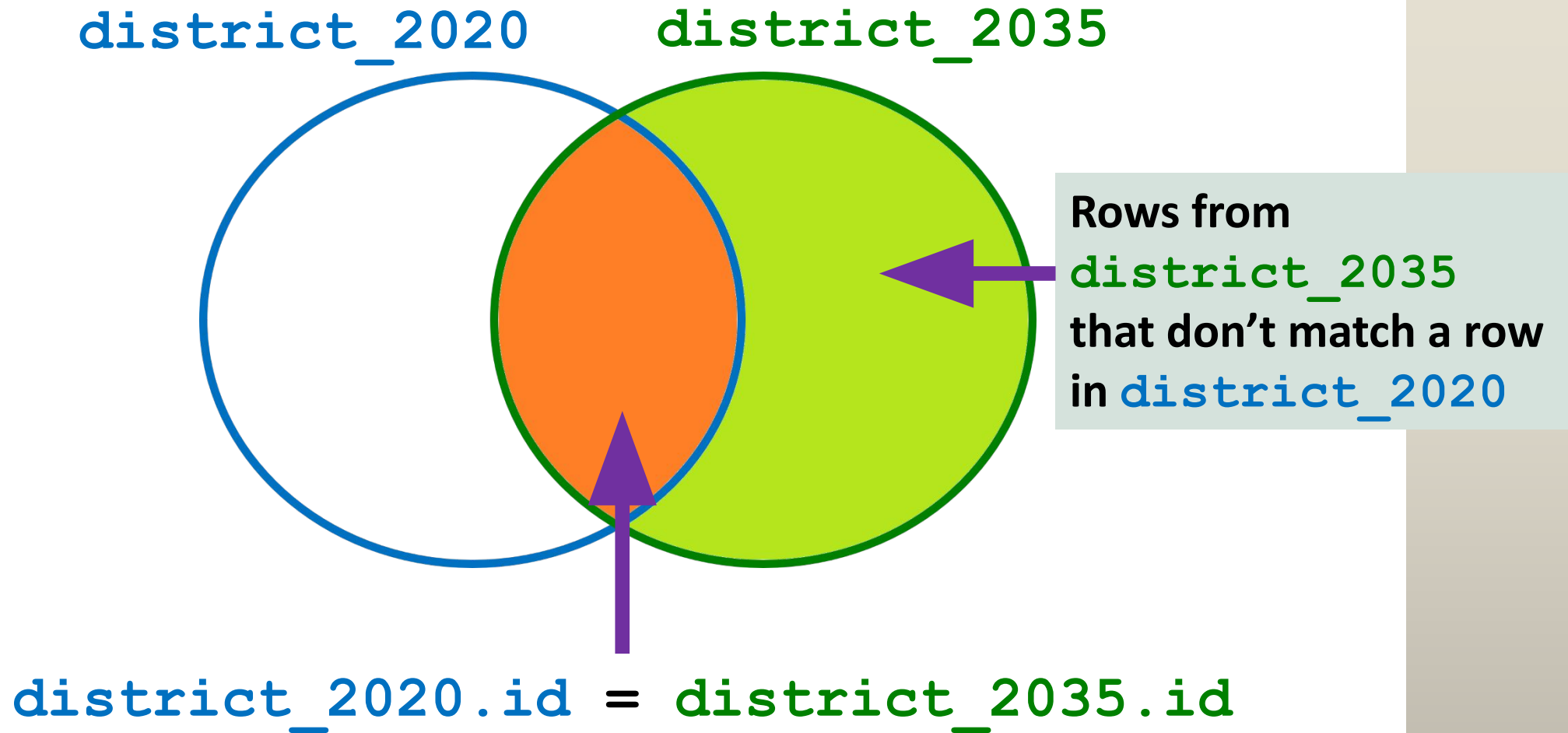
```
SELECT *  
FROM chap_07_examples.district_2020  
LEFT JOIN chap_07_examples.district_2035  
ON district_2020.id = district_2035.id  
ORDER BY district_2020.id;
```

| id | school_2020           | id   | school_2035           |
|----|-----------------------|------|-----------------------|
| 1  | Oak Street School     | 1    | Oak Street School     |
| 2  | Roosevelt High School | 2    | Roosevelt High School |
| 5  | Dover Middle School   | NULL | NULL                  |
| 6  | Webutuck High School  | 6    | Webutuck High School  |

A left outer join returns **all the rows from the left** table and rows **matching the join condition from the right table**. Unmatched values from the right table show up as **NULLs** in the result.

The **NULL** values are returned in the result set where there is not a matching row in the right table.

# A RIGHT OUTER Join



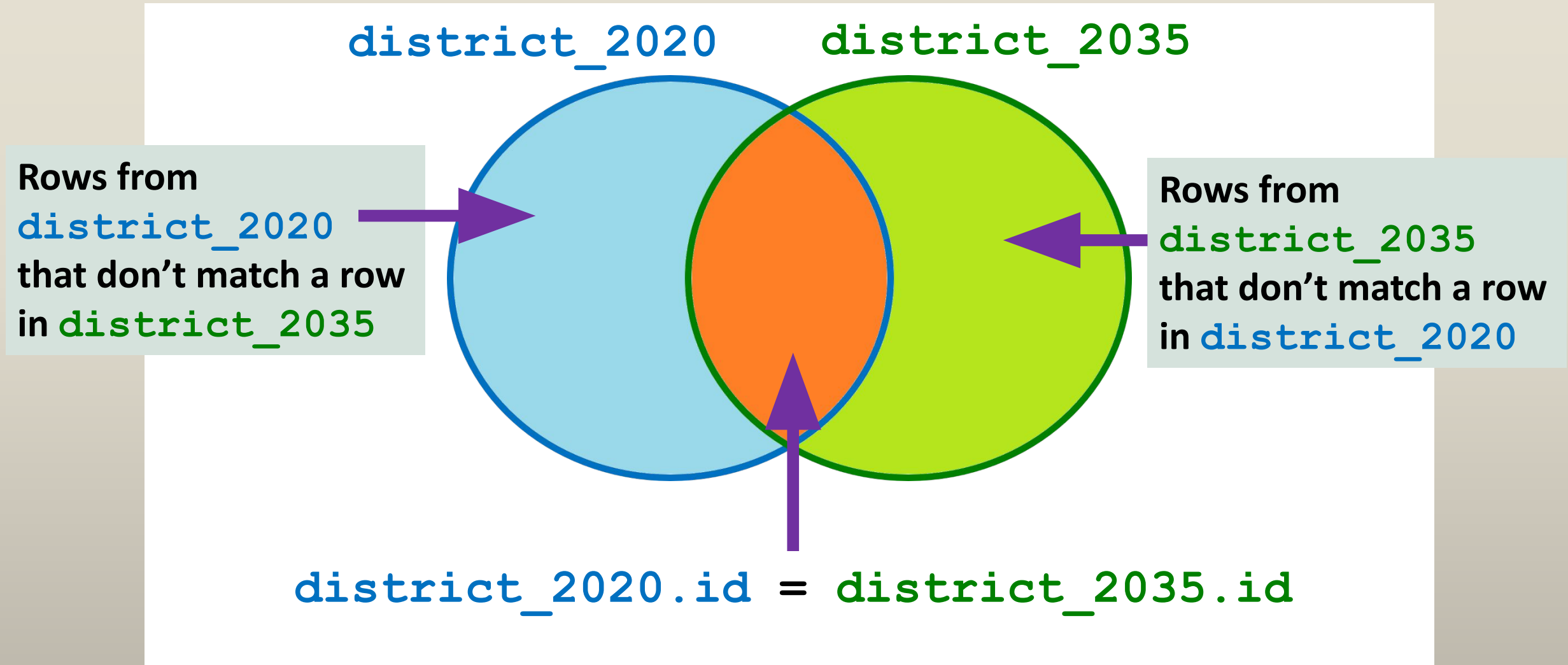
```
SELECT *
FROM chap_07_examples.district_2020
  RIGHT JOIN chap_07_examples.district_2035
  ON district_2020.id = district_2035.id
ORDER BY district_2035.id;
```

A right outer join returns **all the rows from the right table** and rows **matching the join condition from the left table**. Unmatched values from the left table show up as **NULLs** in the result.

| id   | school_2020           | id | school_2035           |
|------|-----------------------|----|-----------------------|
| 1    | Oak Street School     | 1  | Oak Street School     |
| 2    | Roosevelt High School | 2  | Roosevelt High School |
| NULL | NULL                  | 3  | Morrison Elementary   |
| NULL | NULL                  | 4  | Chase Magnet Academy  |
|      |                       | 5  | Webutuck High School  |

The **NULL** values are returned in the result set where there is not a matching row in the left table.

# A FULL OUTER Join

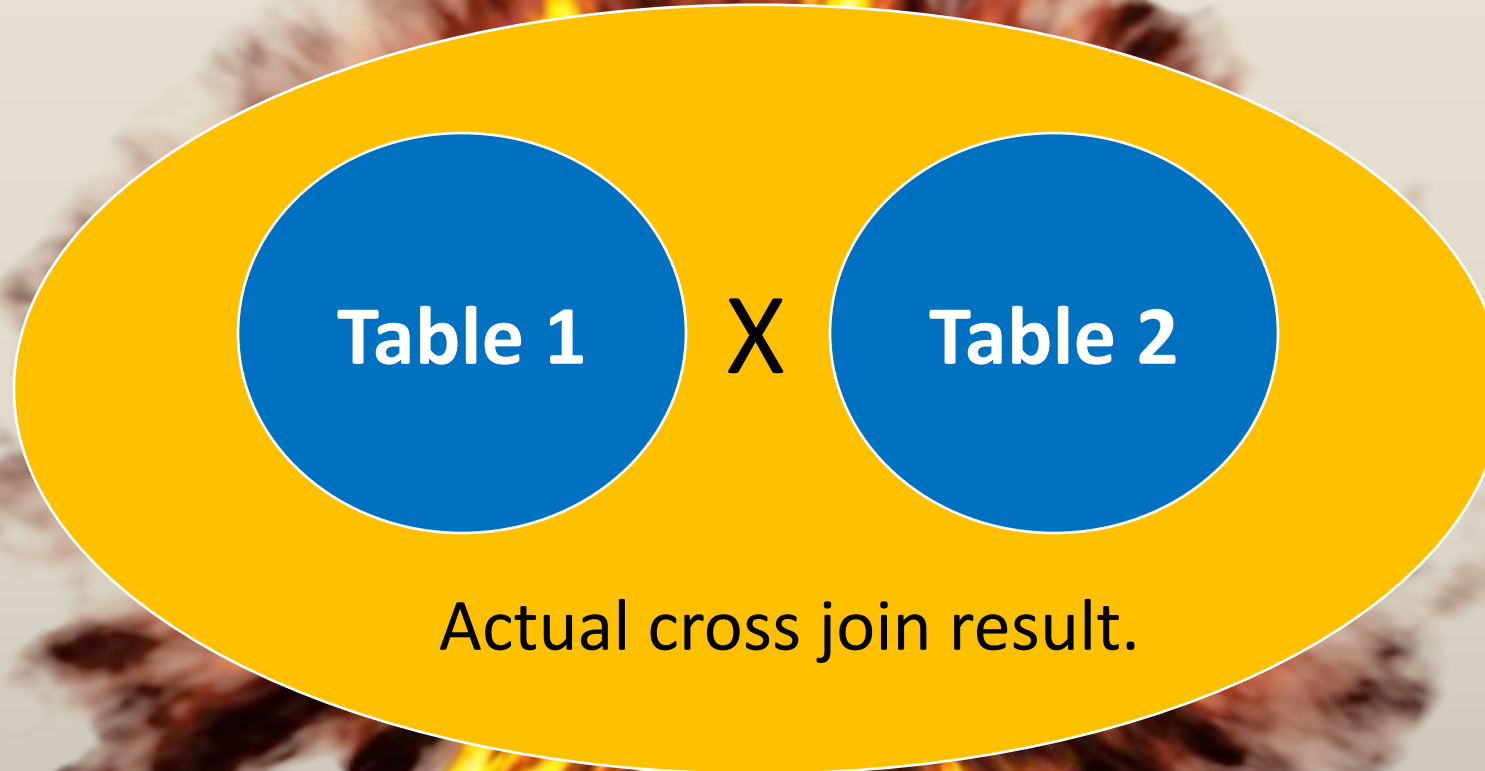


A **FULL OUTER JOIN** gives you all the data that matches from both tables. The unmatched data from the left table shows up as **NULLs**. The unmatched data from the right table shows up as **NULLs**.

```
SELECT *  
FROM chap_07_examples.district_2020  
FULL OUTER JOIN chap_07_examples.district_2035  
ON district_2020.id = district_2035.id  
ORDER BY district_2020.id;
```

| id   | school_2020           | id   | school_2035           |
|------|-----------------------|------|-----------------------|
| 1    | Oak Street School     | 1    | Oak Street School     |
| 2    | Roosevelt High School | 2    | Roosevelt High School |
| 5    | Dover Middle School   | NULL | NULL                  |
| 6    | Webutuck High School  | 6    | Webutuck High School  |
| NULL | NULL                  | 4    | Chase Magnet Academy  |
| NULL | NULL                  | 3    | Morrison Elementary   |

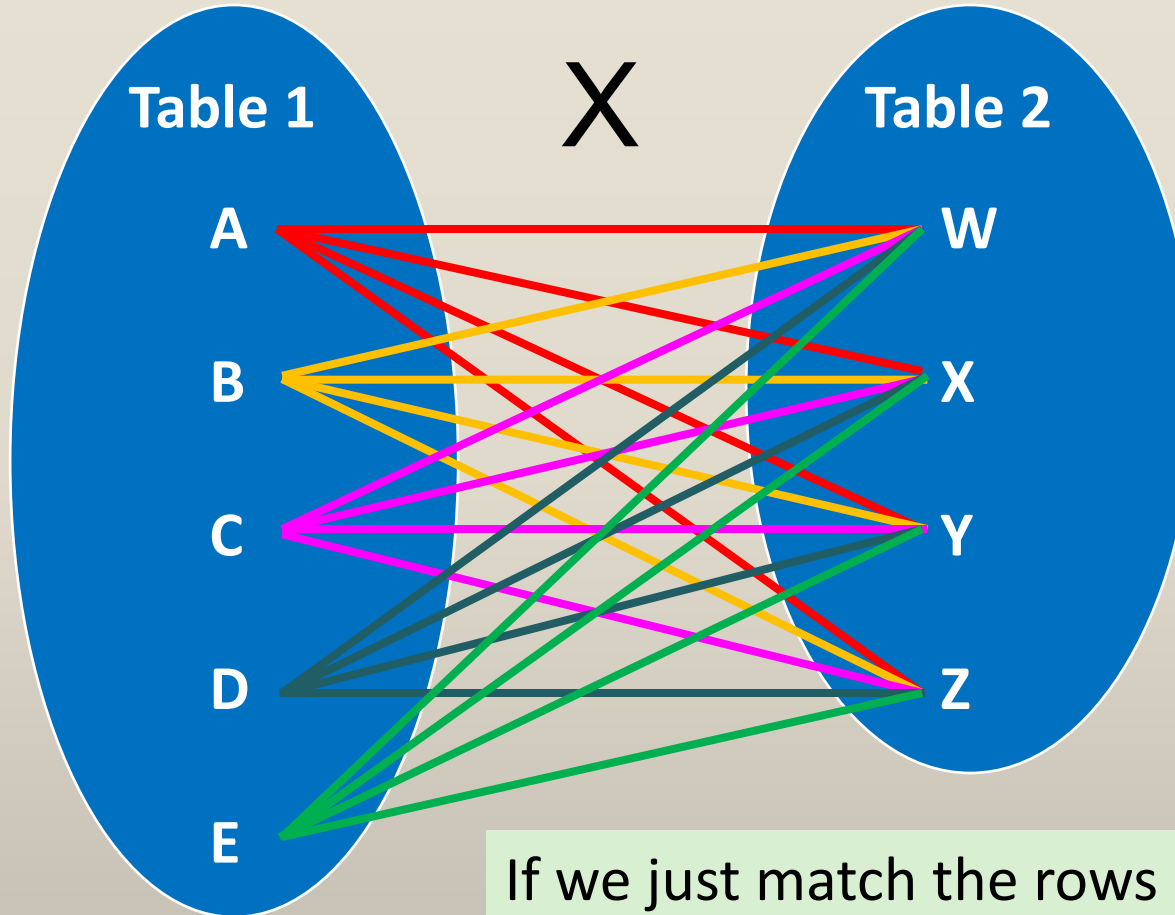
# The CROSS Join



The tables in a **cross join** don't need to have any rows that match a join condition.

I think of the **cross join** as this. The explosion of the amount of data you just received from the query.

# The CROSS Join



## Result

|           |           |           |           |
|-----------|-----------|-----------|-----------|
| <b>AW</b> | <b>AX</b> | <b>AY</b> | <b>AZ</b> |
| <b>BW</b> | <b>BX</b> | <b>BY</b> | <b>BZ</b> |
| <b>CW</b> | <b>CX</b> | <b>CY</b> | <b>CZ</b> |
| <b>DW</b> | <b>DX</b> | <b>DY</b> | <b>DZ</b> |
| <b>EW</b> | <b>EX</b> | <b>EY</b> | <b>EZ</b> |

If we just match the rows pairwise, we come up with  $5 * 4$  pairs.  
That is why it is called a **Cartesian product** of the rows.



# A CROSS Join

```
SELECT *  
FROM chap_07_examples.district_2020  
      CROSS JOIN chap_07_examples.district_2035  
ORDER BY district_2020.id, district_2035.id;
```

```
SELECT *
FROM chap_07_examples.district_2020
CROSS JOIN chap_07_examples.district_2035
ORDER BY district_2020.id, district_2035.id;
```

| id | school_2020           | id | school_2035           |
|----|-----------------------|----|-----------------------|
| 1  | Oak Street School     | 1  | Oak Street School     |
| 1  | Oak Street School     | 2  | Roosevelt High School |
| 1  | Oak Street School     | 3  | Morrison Elementary   |
| 1  | Oak Street School     | 4  | Chase Magnet Academy  |
| 1  | Oak Street School     | 6  | Webutuck High School  |
| 2  | Roosevelt High School | 1  | Oak Street School     |
| 2  | Roosevelt High School | 2  | Roosevelt High School |
| 2  | Roosevelt High School | 3  | Morrison Elementary   |
| 2  | Roosevelt High School | 4  | Chase Magnet Academy  |
| 2  | Roosevelt High School | 6  | Webutuck High School  |
| 5  | Dover Middle School   | 1  | Oak Street School     |
| 5  | Dover Middle School   | 2  | Roosevelt High School |
| 5  | Dover Middle School   | 3  | Morrison Elementary   |
| 5  | Dover Middle School   | 4  | Chase Magnet Academy  |
| 5  | Dover Middle School   | 6  | Webutuck High School  |
| 6  | Webutuck High School  | 1  | Oak Street School     |
| 6  | Webutuck High School  | 2  | Roosevelt High School |
| 6  | Webutuck High School  | 3  | Morrison Elementary   |
| 6  | Webutuck High School  | 4  | Chase Magnet Academy  |
| 6  | Webutuck High School  | 6  | Webutuck High School  |

# The implicit syntax for a **CROSS JOIN**

```
SELECT *  
FROM chap_07_examples.district_2020  
      , chap_07_examples.district_2035  
ORDER BY district_2020.id, district_2035.id;
```

There are no join clauses.

It is very easy to write a query like this and **return much more** data than you anticipate.

No JOIN clause and no WHERE clause.

```
SELECT *  
FROM chap_07_examples.district_2020  
LEFT JOIN chap_07_examples.district_2035  
ON district_2020.id = district_2035.id  
WHERE district_2035.id IS NULL;
```

| id | school_2020         | id   | school_2035 |
|----|---------------------|------|-------------|
| 5  | Dover Middle School | NULL | NULL        |

We can combine an outer join and a where clause to quickly identify values missing from one of the tables.

```
SELECT *  
FROM chap_07_examples.district_2020  
  RIGHT JOIN chap_07_examples.district_2035  
    ON district_2020.id = district_2035.id  
WHERE district_2020.id IS NULL;
```

| id   | school_2020 | id | school_2035          |
|------|-------------|----|----------------------|
| NULL | NULL        | 3  | Morrison Elementary  |
| NULL | NULL        | 4  | Chase Magnet Academy |

Doing the same thing, but  
identifying missing value from the  
left table.

```
SELECT d20.id, d20.school_2020, d35.school_2035
FROM chap_07_examples.district_2020 AS d20
  LEFT JOIN chap_07_examples.district_2035 AS d35
    ON d20.id = d35.id
ORDER BY d20.id;
```

| id | school_2020           | school_2035           |
|----|-----------------------|-----------------------|
| 1  | Oak Street School     | Oak Street School     |
| 2  | Roosevelt High School | Roosevelt High School |
| 5  | Dover Middle School   | NULL                  |
| 6  | Webutuck High School  | Webutuck High School  |

```
SELECT d20.id, d20.school_2020, en.enrollment, gr.grades
FROM chap_07_examples.district_2020 AS d20
  JOIN chap_07_examples.district_2020_enrollment AS en
    ON d20.id = en.id
  JOIN chap_07_examples.district_2020_grades AS gr
    ON d20.id = gr.id
ORDER BY d20.id;
```

Three tables means 2 join conditions.

| id | school_2020           | enrollment | grades |
|----|-----------------------|------------|--------|
| 1  | Oak Street School     | 360        | K-3    |
| 2  | Roosevelt High School | 1,001      | 9-12   |
| 5  | Dover Middle School   | 450        | 6-8    |
| 6  | Webutuck High School  | 927        | 9-12   |

# Understanding the Three Types of Table Relationships

## One-to-One Relationship

- A value in a table maps to 0 or 1 row in a table.

## One-to-Many Relationship

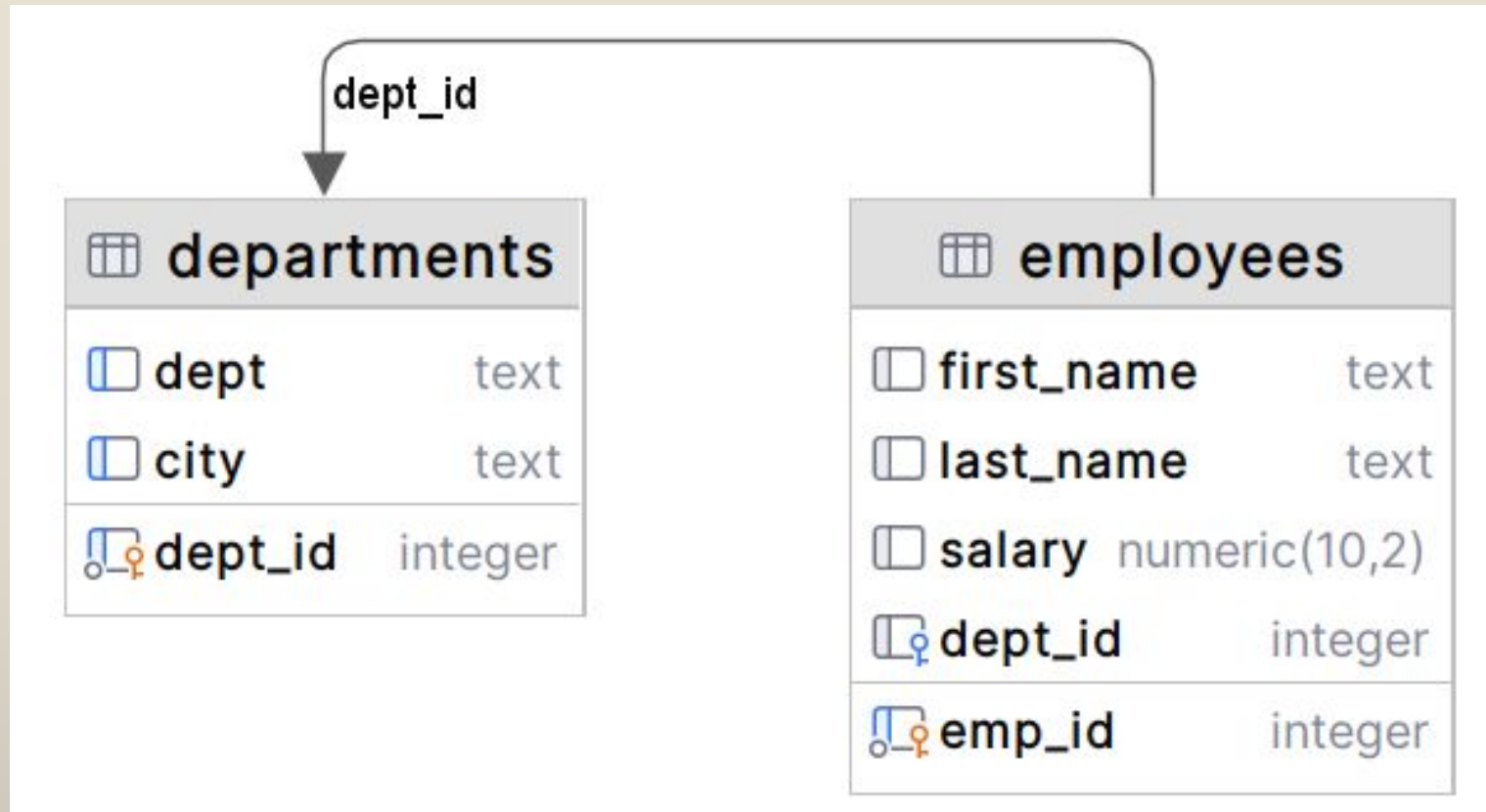
- A value in a parent-table maps to 0 or more rows in a child-table.

## Many-to-Many Relationship

- Many values in one table map to many values in another table.



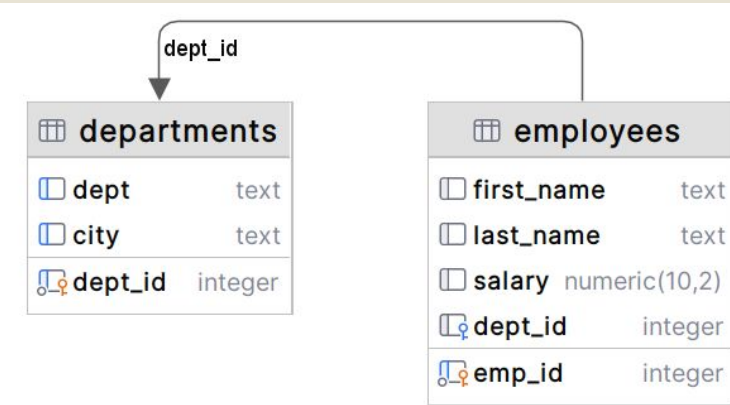
# One-to-many relationship



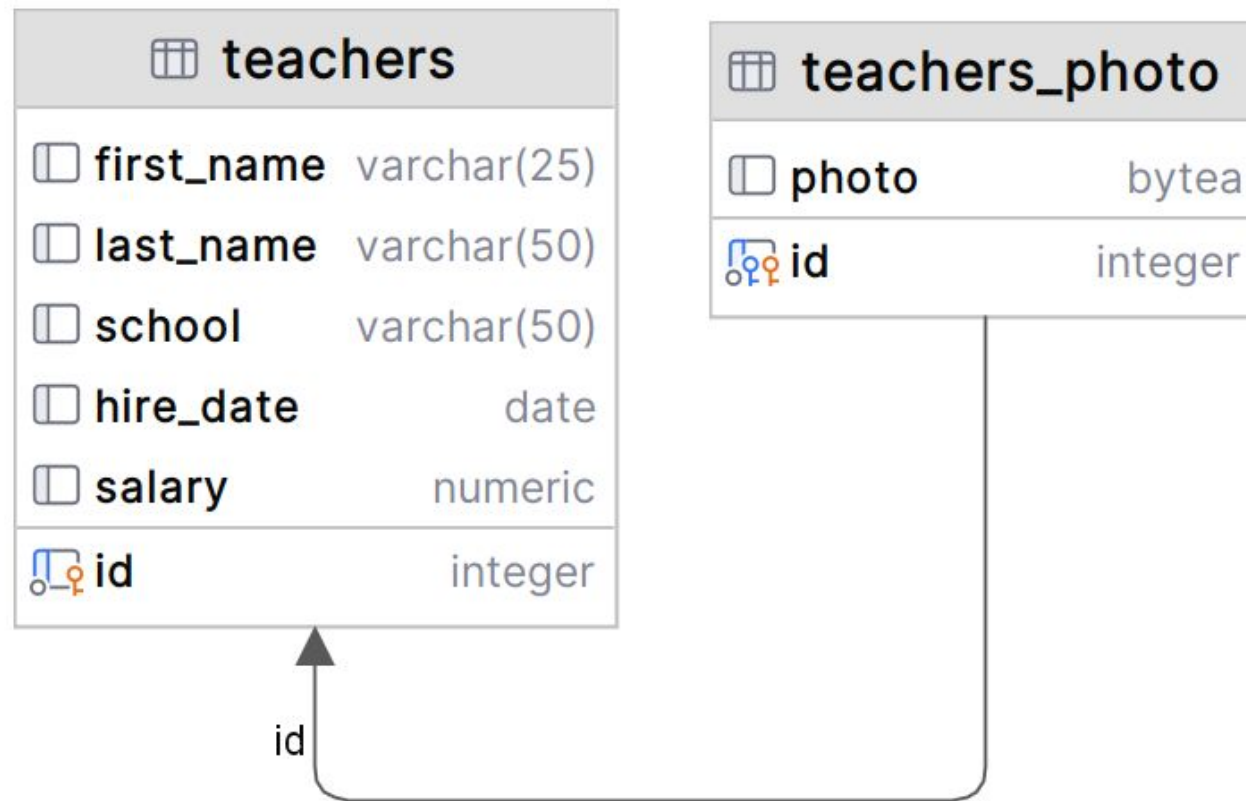
Each **department** can have zero or more **employees**. The mapping is from one **department** to, potentially, many **employees**.

```
CREATE TABLE chap_07_examples.departments (  
    dept_id integer,  
    dept text,  
    city text,  
    CONSTRAINT dept_key PRIMARY KEY (dept_id),  
    CONSTRAINT dept_city_unique UNIQUE (dept, city)  
);
```

```
CREATE TABLE chap_07_examples.employees (  
    emp_id integer,  
    first_name text,  
    last_name text,  
    salary numeric(10,2),  
    dept_id integer  
    REFERENCES chap_07_examples.departments (dept_id),  
    CONSTRAINT emp_key PRIMARY KEY (emp_id)  
);
```



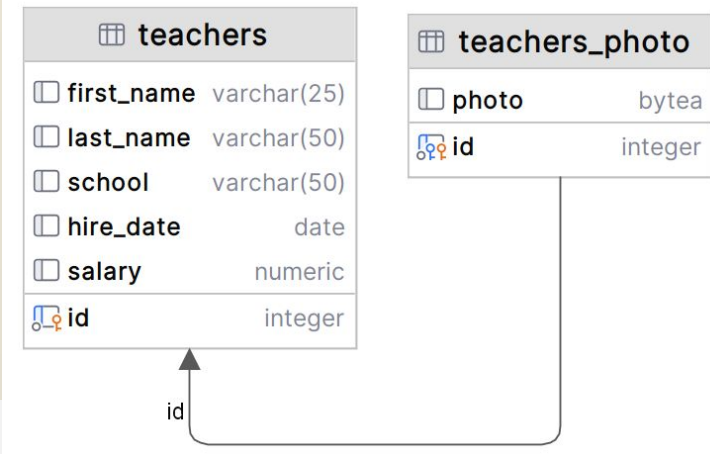
# One-to-one relationship



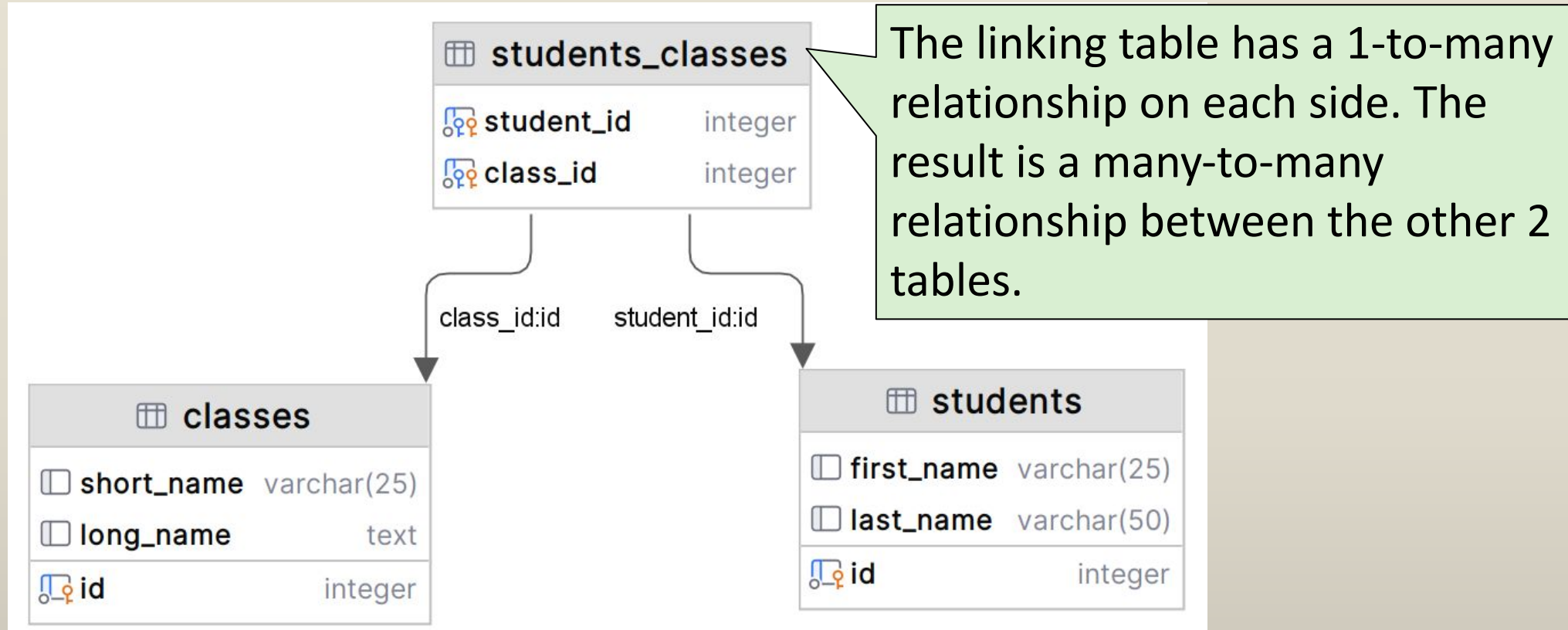
A **teacher** can have a **teacher\_photo**. The photos are stored in a different table. A teacher cannot have more than 1 photo.

```
CREATE TABLE if not exists teachers.teachers (  
    id int generated by default as identity primary key  
    , first_name varchar(25)  
    , last_name varchar(50)  
    , school varchar(50)  
    , hire_date date  
    , salary numeric  
) ;
```

```
CREATE TABLE if not exists teachers.teachers_photo (  
    id int primary key  
    , photo bytea  
    , constraint id_fk foreign key(id) REFERENCES teachers.teachers (id)  
) ;
```



# Many-to-many relationship



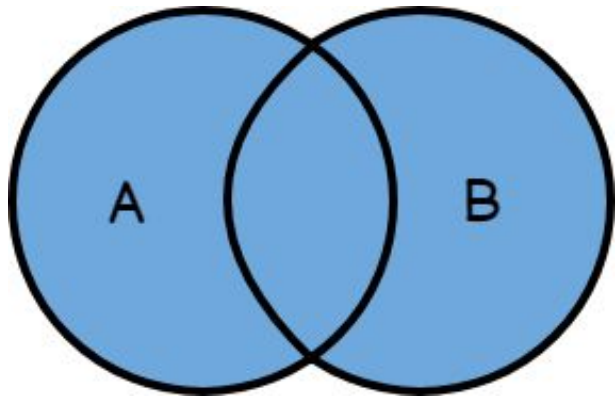
We have many **classes**. We have many **students**. A class can have zero or more students. A student can have zero or more classes.

We have a **linking table** between students and classes to represent the 1-to-many on each side.

```
CREATE TABLE if not exists teachers.classes (  
    id int GENERATED BY DEFAULT AS IDENTITY primary key,  
    short_name varchar(25) NULL,  
    long_name text NULL  
);  
  
CREATE TABLE if not exists teachers.students (  
    id int GENERATED BY DEFAULT AS IDENTITY primary key,  
    first_name varchar(25) NULL,  
    last_name varchar(50) NULL  
);  
  
CREATE TABLE if not exists teachers.students_classes (  
    student_id int NOT NULL,  
    class_id int NOT NULL,  
    CONSTRAINT students_classes_pk PRIMARY KEY (student_id, class_id),  
    CONSTRAINT classes_fk FOREIGN KEY (class_id) REFERENCES teachers.classes(id),  
    CONSTRAINT student_fk FOREIGN KEY (student_id) REFERENCES teachers.students(id)  
);
```

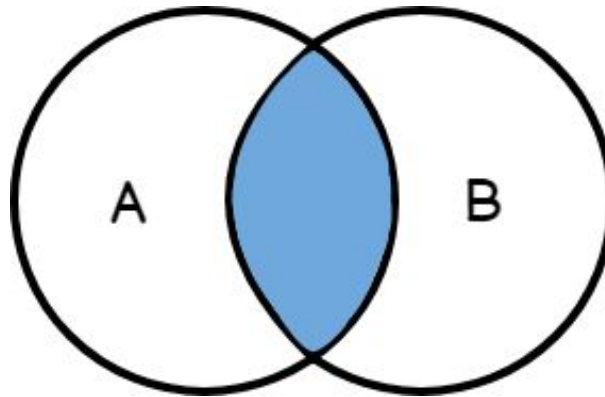
```
select *  
from teachers.students as s  
    join teachers.students_classes as sc  
        on s.id = sc.student_id  
    join teachers.classes as c  
        on sc.class_id = c.id  
;
```

# The Set Operators in SQL



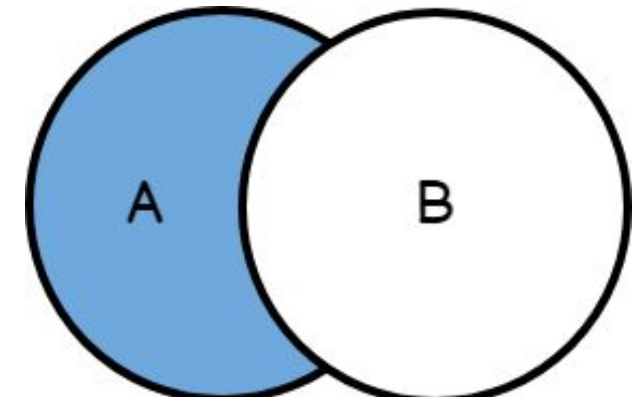
## UNION

Combine the results of 2 queries.



## INTERSECT

What do 2 queries have in common.



## EXCEPT

Remove from the left common entries from the right.



# The syntax for a union operation

```
SELECT_statement_1
UNION [ALL]
      SELECT_statement_2
[UNION [ALL]
      SELECT_statement_3] ...
[ORDER BY order_by_list]
```

**UNION** removes duplicate records (where all columns in the result set are the same), **UNION ALL** does not.

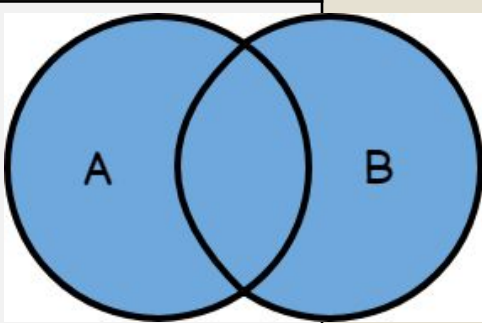
There is a performance hit when using UNION vs UNION ALL, since the database server must do additional work to remove the duplicate rows.

## Rules for a union

- Each result set must return the same number of columns.
- The corresponding columns in each result set must have compatible data types.
- The column names in the final result set are taken from the first SELECT clause.

Column names come from the first query.

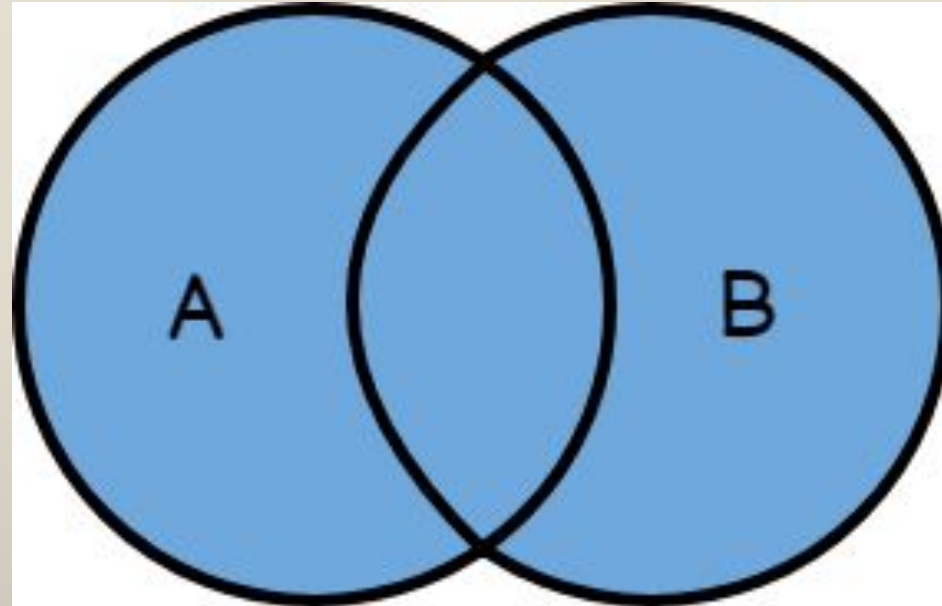
```
SELECT * FROM chap_07_examples.district_2020
UNION
SELECT * FROM chap_07_examples.district_2035
ORDER BY id;
```



| id | school_2020           |
|----|-----------------------|
| 1  | Oak Street School     |
| 2  | Roosevelt High School |
| 3  | Morrison Elementary   |
| 4  | Chase Magnet Academy  |
| 5  | Dover Middle School   |
| 6  | Webutuck High School  |

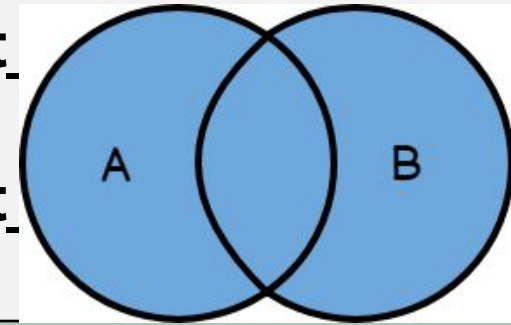
Duplicates are automatically removed with a **UNION**.

```
SELECT *  
FROM chap_07_examples.district_2020
```



```
SELECT *  
FROM chap_07_examples.district_2035
```

```
SELECT * FROM chap_07_examples.district_  
UNION ALL  
SELECT * FROM chap_07_examples.district_  
ORDER BY id;
```



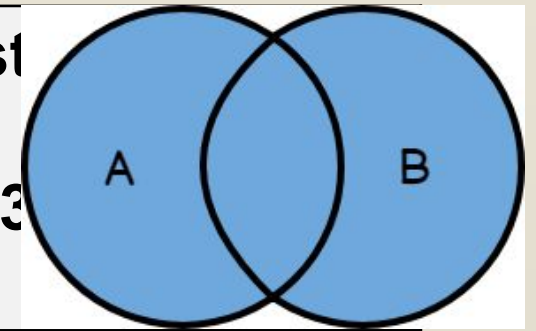
| id | school_2020           |
|----|-----------------------|
| 1  | Oak Street School     |
| 1  | Oak Street School     |
| 2  | Roosevelt High School |
| 2  | Roosevelt High School |
| 3  | Morrison Elementary   |
| 4  | Chase Magnet Academy  |
| 5  | Dover Middle School   |
| 6  | Webutuck High School  |
| 6  | Webutuck High School  |

Duplicates are **NOT**  
automatically removed  
with a **UNION ALL**.

# UNION vs UNION ALL

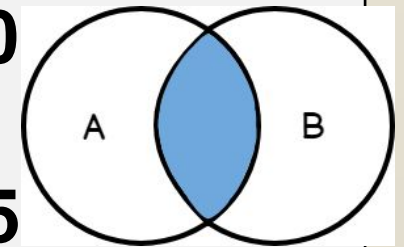
| <b>SELECT * FROM</b><br><b>chap_07_examples.district_2020</b><br><b>UNION</b><br><b>SELECT * FROM</b><br><b>chap_07_examples.district_2035</b><br><b>ORDER BY id;</b> |                       | <b>SELECT * FROM</b><br><b>chap_07_examples.district_2020</b><br><b>UNION ALL</b><br><b>SELECT * FROM</b><br><b>chap_07_examples.district_2035</b><br><b>ORDER BY id;</b> |                       |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|
| id                                                                                                                                                                    | school_2020           | id                                                                                                                                                                        | school_2020           |
| 1                                                                                                                                                                     | Oak Street School     | 1                                                                                                                                                                         | Oak Street School     |
|                                                                                                                                                                       |                       | 1                                                                                                                                                                         | Oak Street School     |
| 2                                                                                                                                                                     | Roosevelt High School | 2                                                                                                                                                                         | Roosevelt High School |
|                                                                                                                                                                       |                       | 2                                                                                                                                                                         | Roosevelt High School |
| 3                                                                                                                                                                     | Morrison Elementary   | 3                                                                                                                                                                         | Morrison Elementary   |
| 4                                                                                                                                                                     | Chase Magnet Academy  | 4                                                                                                                                                                         | Chase Magnet Academy  |
| 5                                                                                                                                                                     | Dover Middle School   | 5                                                                                                                                                                         | Dover Middle School   |
| 6                                                                                                                                                                     | Webutuck High School  | 6                                                                                                                                                                         | Webutuck High School  |
|                                                                                                                                                                       |                       | 6                                                                                                                                                                         | Webutuck High School  |

```
SELECT *, '2020' as year FROM chap_07_examples.dist
UNION ALL
SELECT *, '2035' FROM chap_07_examples.district_203
ORDER BY id;
```



| id | school_2020           | year |
|----|-----------------------|------|
| 1  | Oak Street School     | 2020 |
| 1  | Oak Street School     | 2035 |
| 2  | Roosevelt High School | 2035 |
| 2  | Roosevelt High School | 2020 |
| 3  | Morrison Elementary   | 2035 |
| 4  | Chase Magnet Academy  | 2035 |
| 5  | Dover Middle School   | 2020 |
| 6  | Webutuck High School  | 2020 |
| 6  | Webutuck High School  | 2035 |

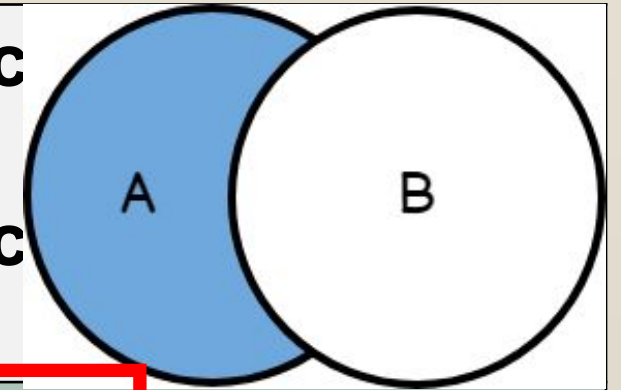
```
SELECT * FROM chap_07_examples.district_2020
INTERSECT
SELECT * FROM chap_07_examples.district_2035
ORDER BY id;
```



| id | school_2020           |  |
|----|-----------------------|--|
| 1  | Oak Street School     |  |
| 2  | Roosevelt High School |  |
| 6  | Webutuck High School  |  |

The 2 result sets have some common rows.  
Show only the common rows.

```
SELECT * FROM chap_07_examples.district  
EXCEPT  
SELECT * FROM chap_07_examples.district  
ORDER BY id;
```



| id | school_2020         |
|----|---------------------|
| 5  | Dover Middle School |

Remove from the left result set all the rows that are also in the right result set.